CryptAlyst — Refactoring Findings

Where the code repeats itself, which design pattern removes the repetition, and what each change buys.

Measured from the tree at commit `a74fbf9` · backend 10,951 lines of Go · web 11,401 lines of TSX · 5,481 lines of CSS

1. The backend has no request layer — 79 handlers rebuild the same preamble

EVIDENCE	COUNT
handlers	**79**
routes registered in one file (`server.go`, 962 lines)	**82**
hand-built `{"ok": false}` responses	**35**
`if s.deps.X == nil` guards	**52**
`r.URL.Query().Get("symbol")`	**23**
`r.URL.Query().Get("interval")`	**18**
manual HTTP-method checks	**11**

```
symbol := strings.ToUpper(strings.TrimSpace(r.URL.Query().Get("symbol")))
if symbol == "" { http.Error(w, "symbol required", http.StatusBadRequest); return }
interval := r.URL.Query().Get("interval")
if interval == "" { interval = "1h" }
horizon := queryInt(r, "horizon", 24)
if horizon < 1 || horizon > 720 { horizon = 24 }
```

Every endpoint repeats parse → validate → default → clamp → check dependency → write the envelope. A new endpoint copies the block and may get any step wrong; one wrong key returns a reply the client cannot parse.

Pattern: middleware chain (method, dependency, timeout) + one typed request struct per endpoint + `okJSON`/`errJSON` for the envelope.

> **Win:** roughly 120 lines of duplicated preamble removed; the response envelope exists once; a wrong method returns 405 instead of 200 with an error body.

2. `Deps` is a god-struct; `Server` is a one-field wrapper around it

```
s.deps.Turtle   36      s.deps.Engine   18      s.deps.Agents   10
s.deps.Market   26      s.deps.Alerts   18      s.deps.Perf      8

type Server struct { deps Deps }
```

No handler declares what it actually needs, so nothing can be constructed in isolation. `go test` passes while covering almost no handler behaviour, because building `Deps` needs the whole world.

Pattern: interface segregation — `BarsSource`, `ModelRunner`, `AlertSink`; handlers declare their own dependencies.

> **Win:** handlers become testable with fakes; a missing dependency becomes a compile error for that handler instead of one of 52 runtime checks.

3. The forecast pipeline is copy-pasted 16 times

Sixteen call sites fetch bars, then each applies its own length check, model call, fan construction and response assembly. The thresholds diverge as a result: 120+horizon in one place, 60 in another, 18 for thirty-second frames.

Pattern: pipeline — `bars → validate(interval) → model → decorate(fan, whiskers) → render`, with the policy in one table keyed by interval.

> **Win:** one definition of “enough data”. A new frame becomes a row in a table, not a search through sixteen call sites.

4. The client has 41 fetch sites in 19 files; `api.ts` is bypassed

EVIDENCE	COUNT
direct `fetch('/api…')` in components	**41**
files calling the API directly	**19**
hand-rolled load / poll / error state per panel	**all**
`setInterval` sites	**30**

Each panel re-implements loading, error and polling, and they disagree: some surface errors, some log them, some leave the previous value on screen so a failed refresh looks like a live one.

Pattern: repository (one typed function per endpoint) + a resource hook owning load, poll, abort and error.

> **Win:** thirty intervals become one hook; every panel gains the same error surface; a stale panel becomes visibly stale.

5. `ForecastLabPanel` is 1,167 lines holding nine sections

Parameters, live engine, data, history, model list, forecast chart, forecast table, ranking, backtest and inputs share one component and one state object.

Pattern: one component per section; context for the three values that cross sections (symbol, interval, method).

> **Win:** last night's model-list redesign touched four separate places in that one file. With sections it would have touched one.

6. Three chart implementations

FILE	LINES	NAVIGATION
`PriceChart.tsx`	**847**	lightweight-charts
`ForecastChart.tsx`	**426**	hand-written
`LiveForecastChart.tsx`	**≈380**	hand-written

Two of them reimplement zoom, pan, reset, crosshair and full screen. The axis defect was one line moved in one copy and missed in the other.

Pattern: adapter — one `ChartSpec` (candles, overlay series, bands) and `PriceChart` as the only renderer.

> **Win:** navigation implemented once; the funnel and branches become overlay series instead of a second canvas.

7. `PriceChart` takes 48 props; `tools` is threaded through 14 components

`tools: ToolsMsg | null` is passed down fourteen layers so a leaf can render a glossary tooltip. The chart's prop list grows by a boolean and a colour for every new study.

Pattern: context for the glossary; one `chartConfig` object for studies.

> **Win:** adding a study stops rippling through seven files; the component's contract becomes readable.

8. `styles.css` is 5,481 lines and 1,260 rules

An `audit-css` gate exists (238 checks) — a guard that exists because a single stylesheet this size cannot be reasoned about. There is no shared vocabulary for spacing or colour, which is why a type scale could not be introduced without changing how the product looks.

Pattern: tokens for spacing, colour and elevation (not font sizes), plus per-domain stylesheets.

> **Win:** specificity fights stop; the audit shrinks to naming discipline.

9. Model lists exist in three places

The backend registry, the GPU sweep grid, and the UI's `MODELS` array. The UI list is hand-maintained, so it can offer a model the platform cannot compute and will silently miss any new one.

Pattern: `GET /api/models` describing name, family and availability; the UI renders what exists.

> **Win:** the picker cannot drift from the engine.

10. `liveState` is a forty-field struct rendered into `map[string]any`

The engine builds untyped maps at the boundary; the `trace: null` crash that blanked the panel came from a key whose type nobody declared.

Pattern: typed snapshot structs with explicit JSON tags; no `map[string]any` at the boundary.

> **Win:** the compiler catches the class of bug the browser found instead.

Order of work, and what each step risks

#	ITEM	BLAST RADIUS	WHY THIS POSITION
1	HTTP layer	backend only	largest duplication, no visible change, easiest to revert
2	Resource hook	UI internals	removes thirty timers, fixes silent error handling
3	Forecast pipeline	backend	ends the threshold-divergence class of bug
4	Interface segregation	backend	unlocks real tests for 79 handlers
5	Split the panel	one file	makes every later UI change cheaper
6	One chart	visible	last, because it changes what is on screen

Items 1–4 change no pixels. Items 5–6 do, and should not be committed alongside them.